Doc3 — Solution Architecture Document: HabbitQuest

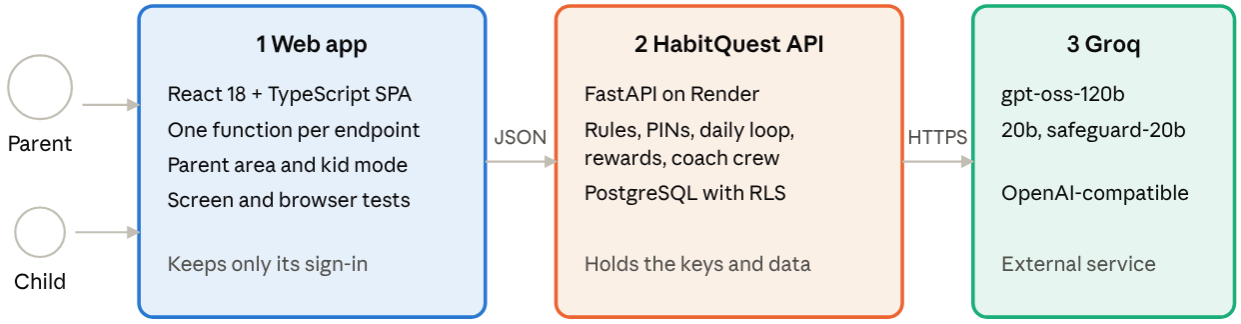
Nov 3, 2026 · @Rahul

1. Architecture overview and goals

HabbitQuest is a web app backed by one API: the React app in the browser shows the game, and the HabbitQuest API (FastAPI) owns every rule, all family data in PostgreSQL and the AI helpers, so the model keys and the data stay on the server.

Three tiers: the game in the browser, rules and data behind one API

HabbitQuest at a glance



Without tier 3 the game still works; AI features fall back to keywords or built-in lines.

architecture at a glance · 3 tiers

Architecture goals

1. **Child safety by construction:** word lists and a safety model check every text a child reads or writes and every parent message to the coach; no AI path can change family data until a grown-up accepts it.
2. **The database enforces privacy:** row-level security means a query only ever sees the caller's family, even if the code forgets a filter.
3. **The game works without AI:** AI failures degrade to keyword matching or built-in lines.
4. **Low AI cost:** open-weight models on Groq, small agents for small jobs, cached buddy lines, a daily AI limit per family.
5. **Testable:** 383 backend tests at 100% coverage, 67 frontend tests and 2 browser tests run on every change.
6. **Swappable provider:** any OpenAI-compatible endpoint, chosen by settings.

2. Architecture drivers

Safety, parent control and cost shaped the architecture more than scale did; a v1 family generates a few kilobytes of data a day.

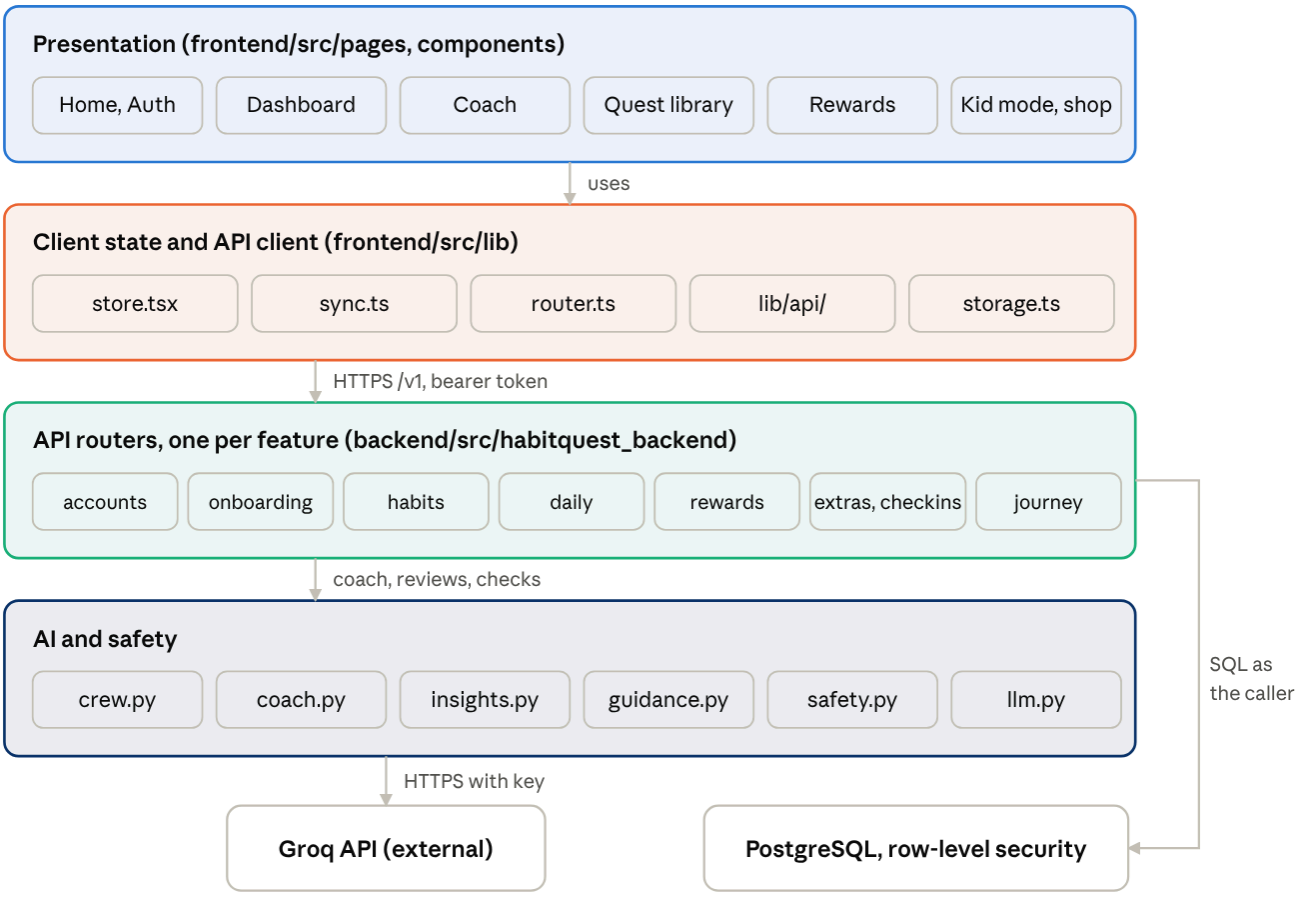
Driver	Source	Architectural response
Children 3-13 must never see harmful text	BR-06, SR-01 to SR-12	Word lists plus a safety model; kid-reads checks and the coach's safety reviewer fail closed
AI must not act without the parent	AI-07, AI-12	Agents only save suggestions; the API applies one when a grown-up accepts it (with the PIN, if set)
API key must stay private	NFR-06	All model calls run inside the API; the browser never sees a key
Game must stay responsive and work without AI	NFR-01, NFR-04	One short database transaction per request; AI calls have deadlines and fallbacks
AI must be affordable	NFR-13, Doc1 business model	Groq open-weight models; lean prompts; small model for high-volume calls
Groq free tier: about 8,000 tokens a minute per model	Constraint	Small agents on the 20b model, only the planner and fit check on 120b; keyword fallback
Advice must be grounded	BR-05	Retrieval over a curated library with inline citations
Children's privacy law (COPPA, GDPR-K)	Doc1 compliance	Labels instead of names to AI; row-level security; export and deletion built; consent still to add
Families must never see each other's data	NFR-08	PostgreSQL row-level security on every family table; separate caregiver and kid database roles

3. Logical architecture

The API is the heart of the system: one FastAPI router per feature, and each request is one database transaction run as the caller, so PostgreSQL's row-level security applies to every query.

The API owns every rule; the browser only shows and asks

Logical layers and their main modules



logical layers · 4 layers, 24 modules

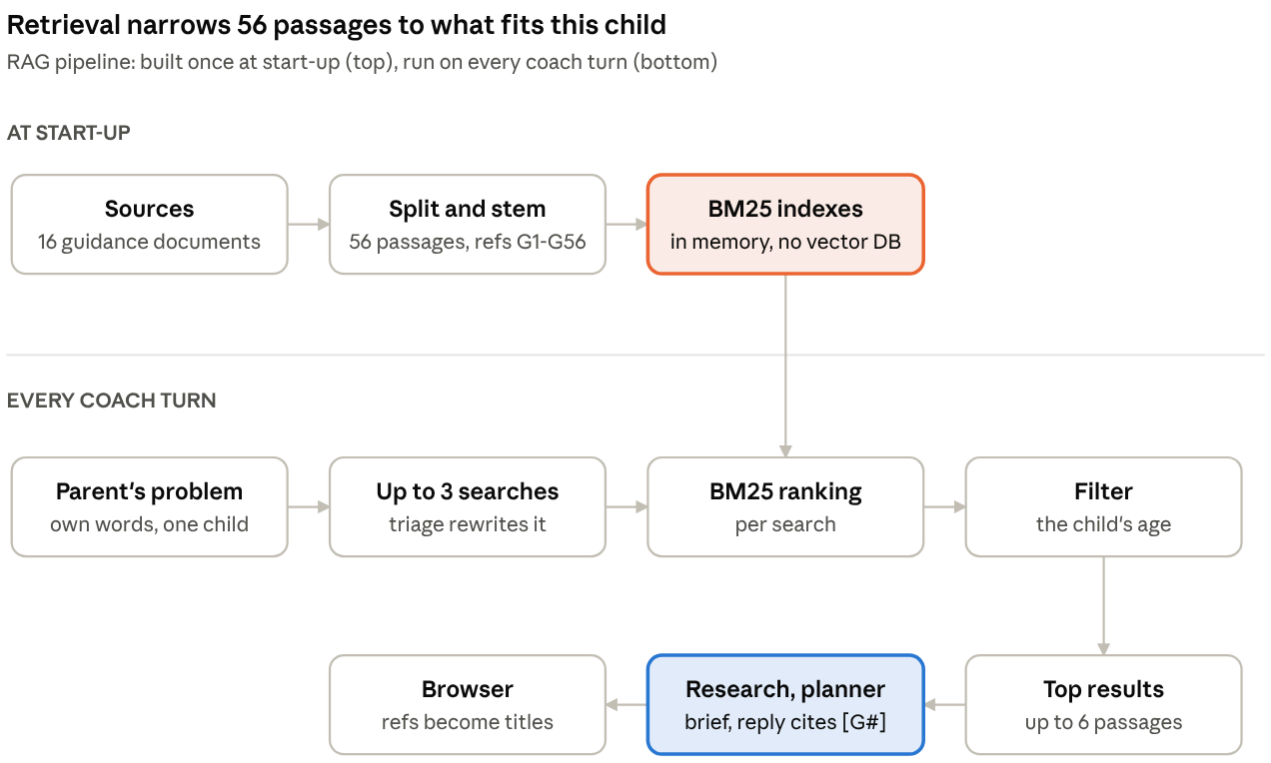
The browser only shows and asks; every rule (age and tier checks, points, boosts, PINs, safety) runs on the server, so no client can bypass it.

Layer	Responsibility	Lives in
Presentation	Pages and components for the parent area and kid mode	frontend/src/pages, components

Layer	Responsibility	Lives in
Client state and API client	App state, PIN prompt, routing; one typed function per endpoint; waiting for a sleeping server	frontend/src/lib, lib/api
API routers	One module per feature: accounts, onboarding, habits, daily, rewards, extras, check-ins, journey, coach, insights	backend/src/habitquest_backend
AI layer	Coach crew (LangGraph), single-agent coach, planner, reviewer, buddy lines, guidance search, safety	crew.py, coach.py, insights.py, guidance.py, safety.py
Data	PostgreSQL schema hq: 22 tables, row-level security, triggers, roles hq_api and hq_kid	backend/db/migrations

4. AI agent architecture

The coach is a crew of small agents in a fixed LangGraph workflow (`crew.py`); retrieval is local BM25, so the only external dependency is the model itself. `COACH_ENGINE=agent` keeps the older single tool-using agent (`coach.py`) for comparison.



RAG pipeline · index lane and query lane

Lexical BM25 was chosen over embeddings for v1: the corpus is small (56 passages from 16 guidance documents), results are deterministic and testable, and there is no embedding cost or vector database to run.

Agents and model routing

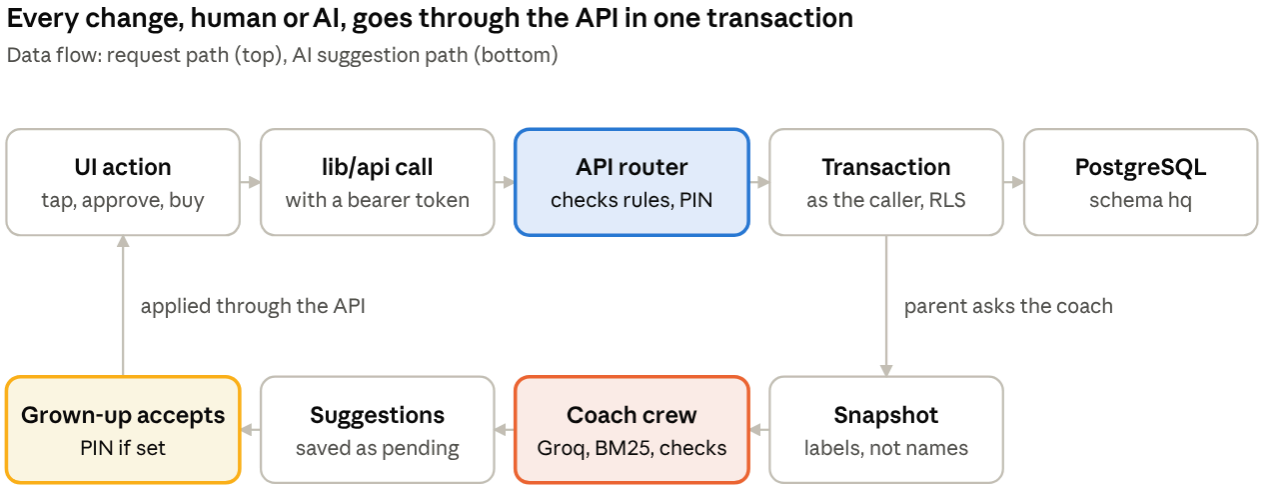
Agent	Model	Pattern	Output
Crew: triage	gpt-oss-20b	One structured call	What the parent needs, up to 3 searches, or one question
Crew: research	gpt-oss-20b	BM25 per search, then one structured call	A brief citing up to 6 passages
Crew: progress analyst	gpt-oss-20b	Counts done in code, then one structured call	Up to 3 patterns
Crew: planner	gpt-oss-120b	One structured call; habit ids limited to an enum of what fits	Reply, up to 3 cards, plan changes
Crew: fit check	gpt-oss-120b	Sees titles only, never the planner's reasons	Poor fits to drop

Agent	Model	Pattern	Output
Crew: safety reviewer	gpt-oss-safeguard-20b	Checks the reply against the coaching rules; fails closed	Safe, or the rules broken
Quest planner, weekly reviewer	gpt-oss-120b	One structured call each, on facts computed in code	Summary, tips and changes; headline, notes and patterns
Buddy	gpt-oss-20b	One text call, then the safety check	One line or a built-in fallback
Parent risk check, kid-text classifier	Safety model	One structured call after the word lists	Risk or not; category

How the crew stays accurate: strict JSON schemas the provider enforces (the planner's habit id is an enum of what fits this child, a citation an enum of the passages found); code does the counting and the models only interpret; one revision when the fit check or the safety reviewer objects, after which whatever still fails is dropped. The single-agent engine keeps its read-only tools (`research` , `recommend_habits` , `propose_habit_change` , `get_quest_history`).

5. Data architecture

All family data lives in PostgreSQL (schema `hq` , 22 tables). Each request runs in one transaction as the caller: the API sets the database role (`hq_api` for caregivers, `hq_kid` for kid tokens) and the caller's id, and row-level security limits every query to that family.



data flow · state path and AI suggestion path

AI output never changes a plan directly: it is saved as a suggestion, and only a grown-up's accept (with the PIN, if set) applies it, with the plan rules checked again.

Where each kind of data lives

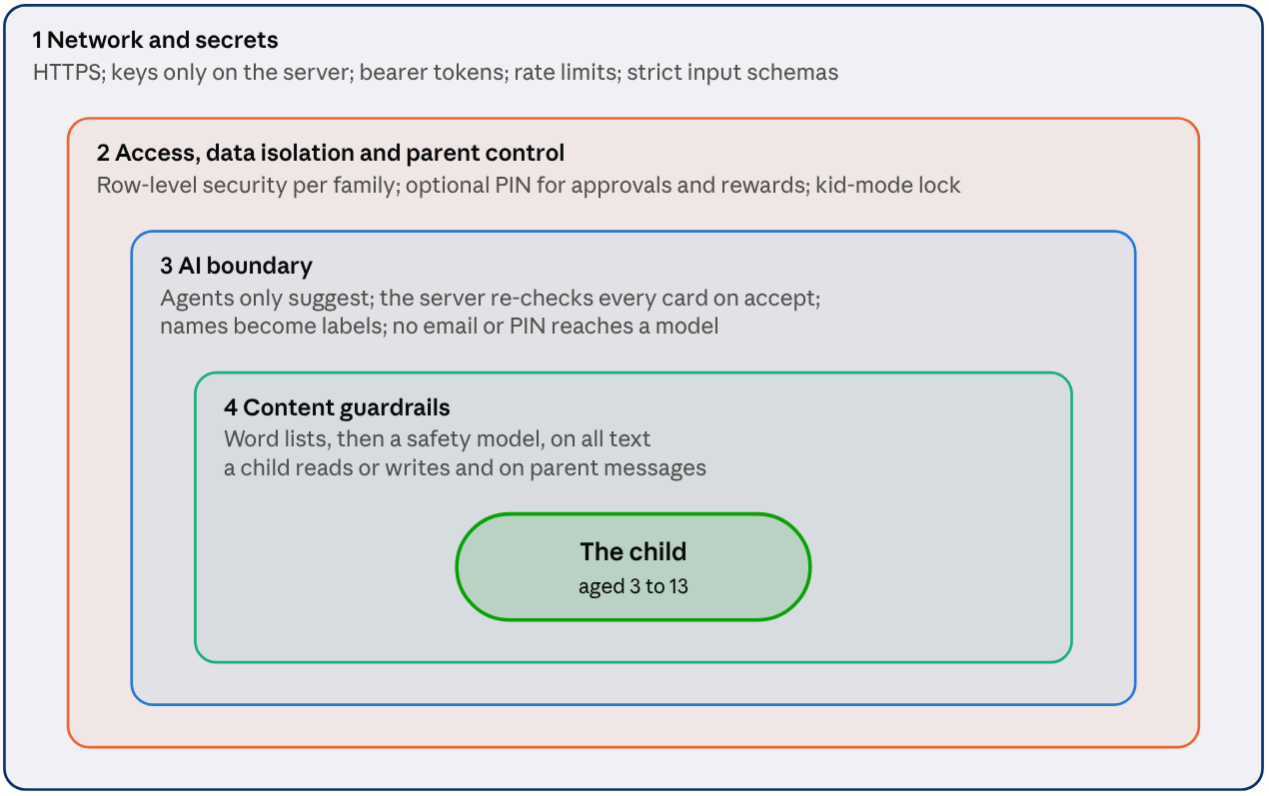
Data	Today	Before public launch
Families, children, plans, completions, points	PostgreSQL with row-level security; the points ledger is append-only and a trigger keeps balances at 0 or above	A paid managed database with backups (the free one expires on 28 November 2026)
Sign-in	Google ID tokens checked on the server, then our own signed bearer tokens (8 hours; development login 1 hour)	Turn off the development login; publish the Google consent screen
PINs	scrypt hashes on the server; attempts rate-limited	No change
Coach history	coach_messages, kept 90 days	No change
Knowledge base, catalog	guidance.json and the seeded catalog in the repository; custom habits in the database	Expert review of the guidance
Safety alerts	Flagged check-ins, shown to grown-ups until marked done	Human review of flagged cases

6. Security, privacy and safety architecture

No single control protects a child on its own; each layer assumes the one outside it can fail.

Four layers of defence stand between the outside world and a child

Security, privacy and safety controls, outermost first



defence in depth · 4 layers

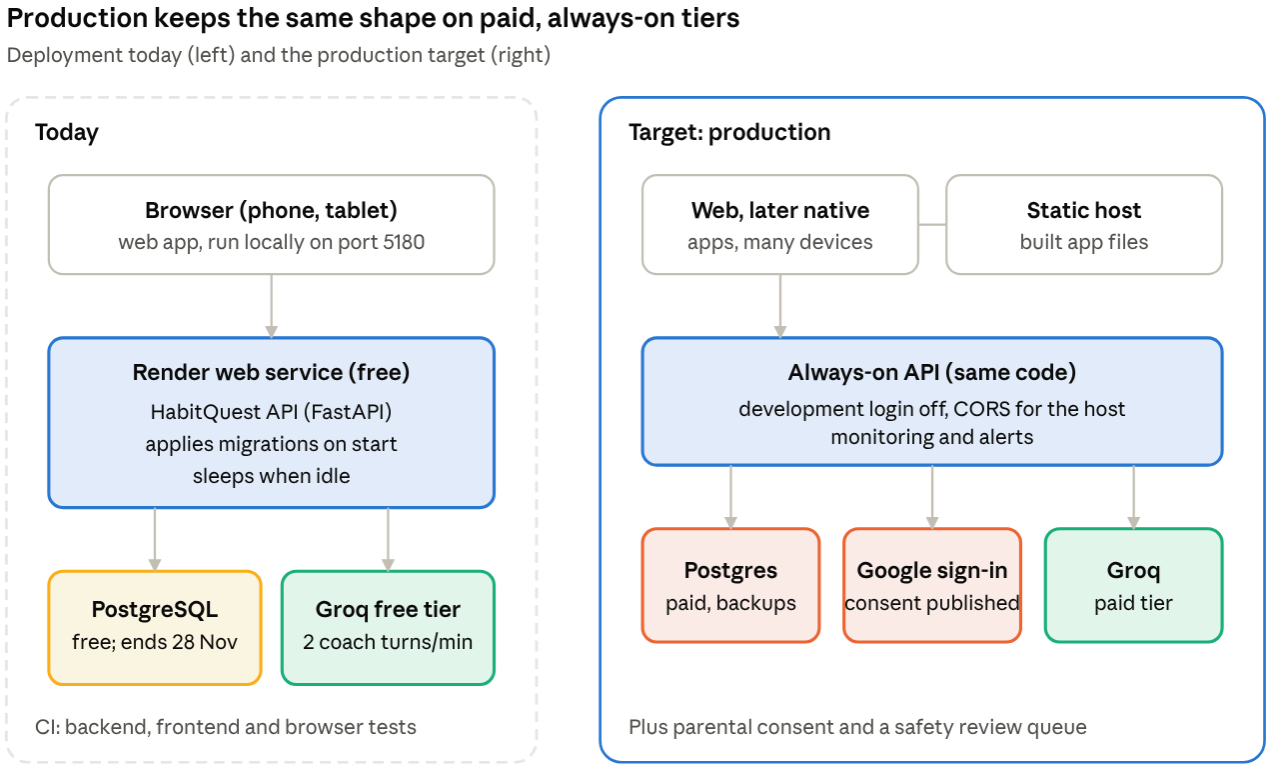
Gaps to close before public launch

Gap in the prototype	Status	What is left
Passwords and PINs in the browser	Closed: Google sign-in, server-side scrypt PIN hashes, rate-limited attempts	Turn off the development login
Agent API had no authentication	Closed: every endpoint needs a caregiver or kid token; sign-in and PIN rate limits; a daily AI limit per family	Abuse monitoring
Data only on one device	Closed: PostgreSQL with row-level security on a hosted database	Paid database with backups and encryption at rest; choose the hosting region
No consent or deletion	Partly closed: family export and deletion, leaving a family, deleting a child, a privacy page naming the AI provider	COPPA verifiable parental consent

Gap in the prototype	Status	What is left
Guardrail word lists are short	Partly closed: a safety model behind the lists, a coach safety reviewer that fails closed	Red-team test set; human review of flagged cases

7. Deployment architecture

The API runs on Render (a free web service and free PostgreSQL today) and starts with `scripts/render_start.sh`, which applies pending migrations and the catalog seed; the frontend is a static Vite build that any static host can serve.



deployment · today vs production target

CI runs on GitHub Actions for every change: backend checks (ruff, mypy, pytest at 100% coverage on PostgreSQL 17), frontend checks (vitest, tsc, build) and browser tests (Playwright against the real API and database).

Steps to production

1. Upgrade the database to a paid plan before 28 November 2026 (the free one is deleted with its data) and turn on backups.
2. Move the API to an always-on instance; set `AI_MODE=openai`, the model settings and `COACH_ENGINE=crew`; move to a Groq paid tier.

3. Host the built frontend (for example a Render static site); add its address to `CORS_ORIGINS` and to the Google client.
4. Turn off `DEV_LOGIN`, publish the Google consent screen, and add monitoring and alerts.
5. Add verifiable parental consent and finish the compliance review.

8. Architecture decisions

Each decision below is recorded with the main alternative and the reason it lost.

ADR	Decision	Alternative considered	Why
01	Superseded by 11. Was: game rules as pure TypeScript functions over one family object	A backend with services per feature	Instant UI, offline play, trivially testable; v1 data is tiny
02	Superseded by 11. Was: agent server as Vite middleware in v1	Separate Express app	One command to run; handler stays portable to any Node host
03	Groq with gpt-oss-120b and gpt-oss-20b	Claude (Anthropic)	Lower cost; same provider as the Wealth Intelligence Copilot; open-weight models
04	LangChain's OpenAI-compatible chat client (was plain `fetch`)	Provider SDK	No vendor lock-in; easy to switch providers
05	BM25 retrieval in memory	Embeddings and a vector database	51 passages; deterministic, free, testable; no extra service
06	Inline [G#] citations mapped to titles in the browser	Provider citation features	Works with any model; titles come from our own data
07	Superseded by 13. Was: one combined `research` tool; family overview in the system prompt	Separate overview, guidance and quest tools	Cut coach calls from about 5 to 3 to fit the 8k tokens-per-minute limit

ADR	Decision	Alternative considered	Why
08	Agents save suggestions; the API applies one when a grown-up accepts it	Agents with write tools	Parents stay in control; no AI path can change data
09	Two guardrail layers: local word checks, then an AI classifier	AI classifier only	Works offline and catches obvious cases instantly; AI covers nuance
10	Strict JSON schemas the provider enforces (pydantic), replacing JSON mode with Zod	Free-text parsing	The decoder cannot produce an invalid answer, and an enum stops invented habits or citations
11	A FastAPI service with PostgreSQL owns all rules and data	Rules in the browser with local storage (01, 02)	Accounts, sync across devices, several grown-ups and server-checked PINs need a server; rules run once, where no client can bypass them
12	Row-level security in PostgreSQL, with caregiver and kid roles	Filtering by family in application code only	A forgotten WHERE clause cannot leak another family's data
13	The coach as a crew of small agents in a fixed LangGraph workflow	One tool-using agent (still available as COACH_ENGINE=agent)	Smaller, cheaper calls; code does the counting; an independent fit check and safety review; scored with eval_coach.py
14	A model risk check on every parent message, after the word lists	Word lists only	Catches unusual wording the lists miss; if the model is busy, the lists have still run

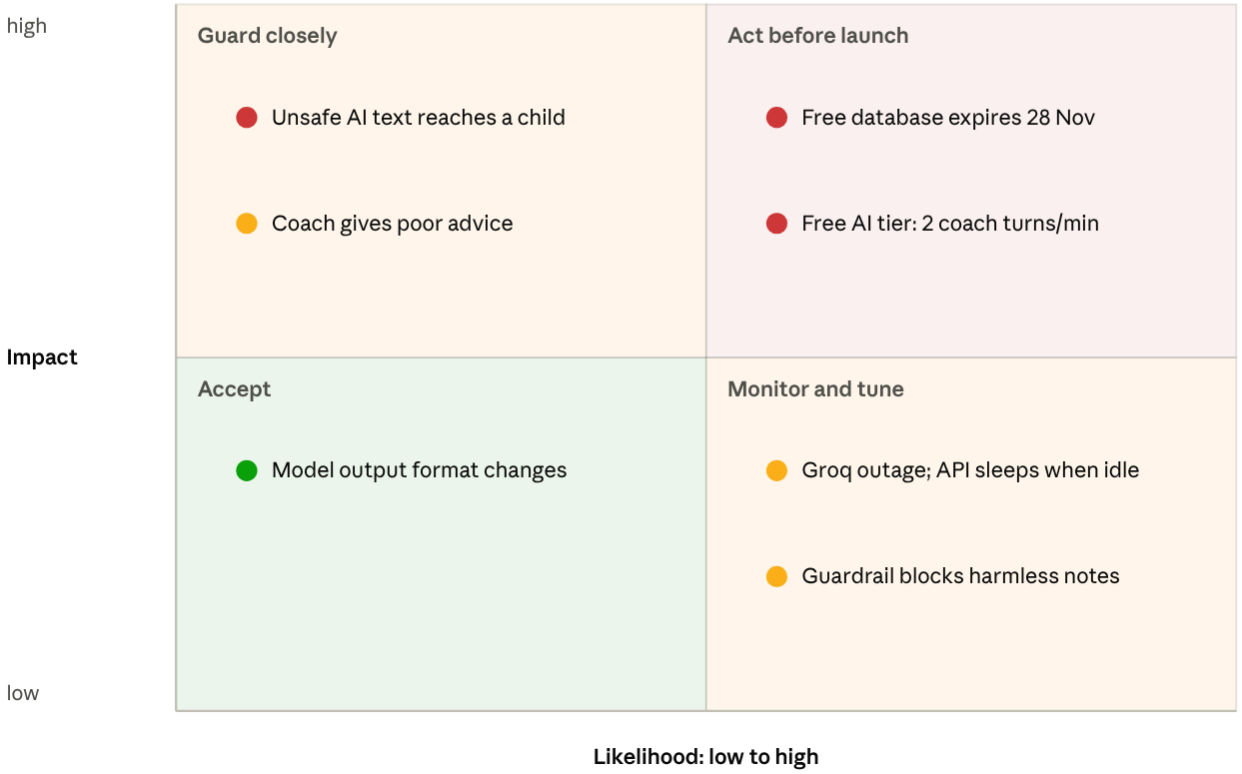
ADR	Decision	Alternative considered	Why
15	An optional family PIN	A PIN required for every family	Families whose children use their own devices do not need one; every guarded action still needs a signed-in grown-up

9. Risks, technical debt and evolution

The architecture is ready for a pilot: accounts, server-side data and authentication are built. The open risks are operational: the free database expires on 28 November 2026, the free AI tier allows about two coach turns a minute, and parental consent is not built yet.

Two risks need action before the pilot: the free database and the free AI tier

Architecture risks placed by likelihood and impact (zones, not scores)



risk matrix · 7 risks in 4 zones

Risk or debt	Mitigation today	Next step
Free database expires on 28 November 2026	Migrations and the seed rebuild the schema, but not the data	A paid database with backups before the pilot

Risk or debt	Mitigation today	Next step
Groq free-tier limits or outage	Small agents, keyword fallback, Groq and Nebius presets	Paid tier; automatic failover to a second provider
Unsafe AI text reaches a child	Word lists and a safety model; fail-closed buddy lines and coach reviewer	Red-team test set; human review queue
Coach gives poor advice	Curated library, citations, schema-limited picks, fit check, eval_coach.py	Expert-reviewed knowledge base; a model judge in the evals
Guardrail blocks harmless notes	Tests for common false positives	Tune lists with consented pilot data
Free API instance sleeps	The app shows "waking up the server" and retries reads	An always-on instance
Abuse of the API	Tokens on every endpoint, sign-in and PIN rate limits, a daily AI limit per family	Monitoring and alerts